Order Statistics

We know that in an array of n elements, we can find the maximum element (or the minimum element) in $O(n)$ time, by iterating through the array once. What if we want to find the median of the array, in what time complexity can we do so? What about finding the ith smallest element in the array?

Selection Problem

We wish to find the median of an array A of size n. Define the **median finding problem** as follows:

Input: An array A of n distinct numbers.

Output: The $\left\lfloor \frac{n+1}{2} \right\rfloor th$ smallest element in A.

Note that usually for even n we define the lower median and upper median separately; but for simplicity, here we will use the term **median** to mean the lower median for even n, and simply the median for odd n.

We wish to solve the median finding problem for any input array A as efficiently as possible. Firstly, we can establish **upper bounds** for this problem, by showing an algorithm for solving the problem. What algorithms can one think of that solve this problem?

- We know that finding the smallest element in an array of size n can be done in $O(n)$. Similarly, we can find the next smallest element in $O(n)$. Continuing as so, the kth smallest element could be found in k passes, taking $O(kn)$ time. For finding the median, this resolves into a $O(n^2)$ time algorithm.
- Obviously, we can do better. Recall, that we can sort the array A in $O(n \log n)$ time. Once sorted, we can simply extract the kth element ($k = \left\lfloor \frac{n+1}{2} \right\rfloor$) to get the median.

Here, we will discuss a **deterministic worst-case linear time** algorithm to find the median of an array A.

This is one of the common situations in computer science, where solving a more general problem is often easier to solve[1]. So, we generalise the problem. Define the **selection problem** as follows:

Input: An array A with n distinct elements, and an integer i, $1 \leq i \leq n$.

Output: The ith smallest element in A.

We will actually solve the generalised selection problem in linear time.

Recursive algorithm

Let A be an array of size n. For some $x \in A$, define $L_x = \{y \in A \mid y < x\}$ be the left partition of A with respect to x, and $R_x = \{y \in A \mid y > x\}$ be the right partition of A with respect to x.

An element $x \in A$ is said to be an **approximate median**, if $|L_x| \geq \frac{n}{4}$ and $|R_x| \geq \frac{n}{4}$. This means, that for an approximate median x, at least some constant fraction (here this fraction is $\frac{1}{4}$) of the elements in the array are larger than x and smaller than x.

[1] This phenomenon is often termed the Inventor's paradox.

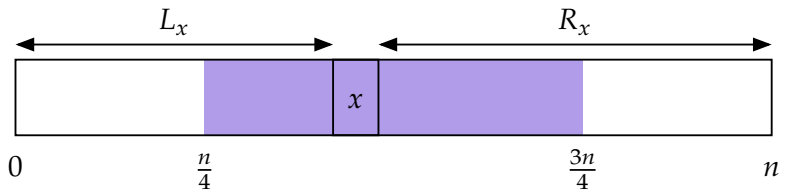


Figure 1: Approximate median x, and L_x and R_x, depicted in a sorted array.

The main idea of the algorithm is as follows: Given an approximate median x, we can partition the array based on x, and recurse in the appropriate subpart of the array.

Partitioning an array A about an element $x \in A$ results in x being moved at its correct position in the sorted array, and all elements less than x (L_x) are placed on the left of x, and all elements greater than x (R_x) are placed on the right of x. We know that we can partition the array in $O(n)$ time, as we do in **quicksort**. We will use the same PARTITION procedure here as well.

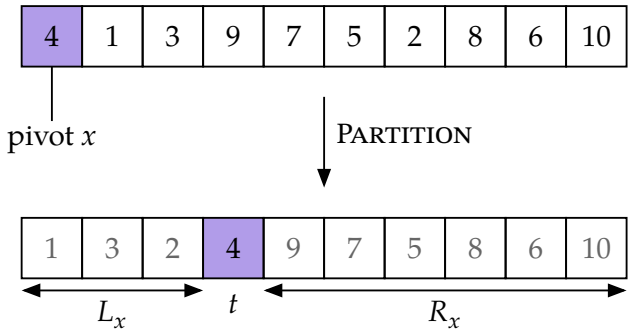


Figure 2: Partitioning an array A based on a pivot x.

Then, we can write the informal pseudocode of the recursive SELECT procedure:

SELECT(A, l, r, i) ▷ *Finds the ith smallest element in the subarray* $A[l..r]$

```
1  x ← FIND-APPROXIMATE-MEDIAN(A, l, r)
2  t ← PARTITION(A, l, r, x)      ▷ t is the absolute position of the pivot x after partitioning the array
3  j ← t − l + 1                  ▷ rank of x within A[l..r]
4  if i = j
5      return x
6  else if i < j
7      return SELECT(A, l, t − 1, i)
8  else
9      return SELECT(A, t + 1, r, i − j)
```

First we start with considering $x \in A$ to be some approximate median in A. (We will come to how we find this later.) Then, considering this element x as a pivot, we partition the array A. Let the position of x in the partitioned array be t, and let j be its relative rank in the $l..r$ subarray. If $i = j$, that is we want to find the j'th element in A, we already know that is x, so we return it.

Otherwise, if $i < j$ then we know that the element we are searching for is less than x, and therefore must be in the left partition of this array. So, we recurse into left part of the array ($A[l..t-1]$), and find the ith smallest element in this subarray. Otherwise, $i > j$, so the element being searched for is on the right of x. We recurse into the right partition ($A[t+1..r]$), and in

this, we want to find the $(i-j)$th smallest element; (since it will be the ith smallest element in the full array $A[l..r]$).

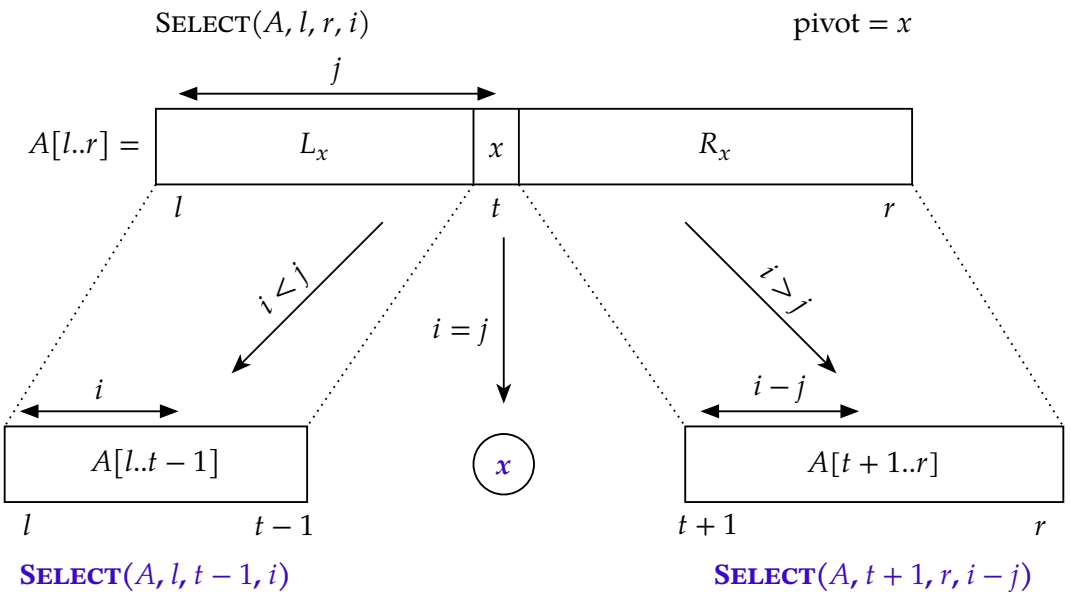


Figure 3: The recursive SELECT procedure

(Note that the SELECT procedure correctly finds the ith order statistic regardless of the chosen pivot. Choosing the pivot element uniformly at random results in the algorithm known as **quickselect**. Here, we wish to make the pivot an approximate median only to improve the worst case running time of the algorithm.)

Finding the pivot

Now, we need to actually find a pivot x which is an approximate median, i.e. at least a constant fraction of the array is to its left and right. By choosing such an element as a pivot, we ensure that some constant fraction of the array is always discarded on each recursive call, thus achieving our desired time complexity (as we will see).

To find such a pivot, we will find the *median of medians* of groups of the array A, which we will show to be an approximate median.

To find an approximate median x in an array A:

1. Divide the n elements in A into $\lfloor n/k \rfloor$ groups of k elements each, and at most one group of the remaining elements.
2. Find the median of each of the $\lceil n/k \rceil$ groups by fully sorting each list and picking the middle element. Call this list of medians M.
3. Call SELECT recursively on M, to find the median of the $\lceil n/k \rceil$ medians found above. This element thus found is x, the pivot for our algorithm.

For our analysis, let us choose $k = 5$ (as depicted in Figure 4). We will justify this choice later.

For this choice of x (the median of medians of the groups), let us try to get a bound on $|L_x|$ and $|R_x|$. We will use it when showing the time complexity of the overall algorithm.

Let $m = \left\lceil \frac{n}{5} \right\rceil$ be the total number of groups.

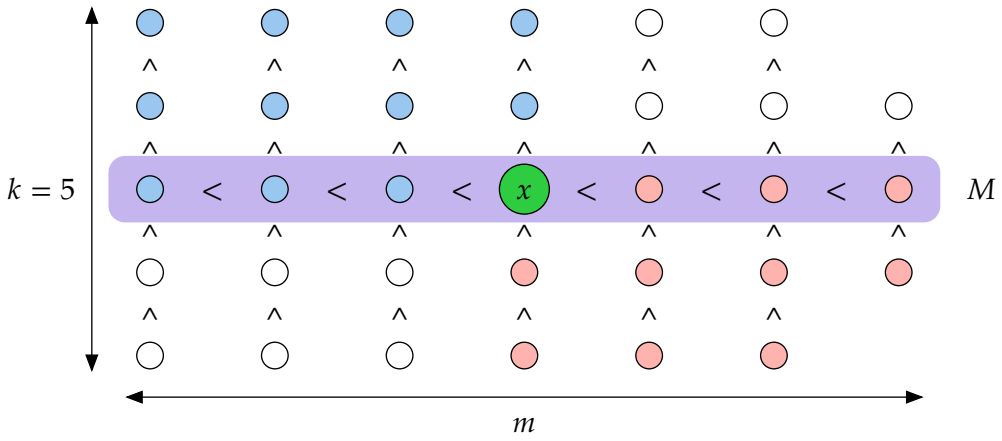


Figure 4: Choosing a pivot x by finding the median of medians. All blue elements are smaller than x and red elements are larger than x.

Observe, that at least $\left\lfloor \frac{m-1}{2} \right\rfloor$ medians are smaller than x. Consider any such group. Each such full group has at least 3 elements which are necessarily less than x, we color these blue. At most one of these groups may be a partial group so let us discard it. Then, we have the following bound on the number of blue elements (which is a subset of L_x):

$$m \geq \frac{n}{5} \Rightarrow \left\lfloor \frac{m-1}{2} \right\rfloor \geq \frac{n}{10} - 1$$

$$|L_x| \geq 3\left(\frac{n}{10} - 2\right) = \frac{3n}{10} - 6$$

Similarly, the same argument also applies to R_x (elements greater than x are colored red in Figure 4).

So, we have the bounds that $|L_x| \geq \frac{3n}{10} - 6$ and $|R_x| \geq \frac{3n}{10} - 6$. Note that for $n \geq 140$, we have $|L_x| \geq \frac{n}{4}$ and $|R_x| \geq \frac{n}{4}$, so x is an *approximate median*.

Time Complexity

SELECT(A, l, r, i)

```
1  x ← FIND-APPROXIMATE-MEDIAN(A, l, r)
2  t ← PARTITION(A, l, r, x)
3  j ← t − l + 1
4  if i = j
5      return x
6  else if i < j
7      return SELECT(A, l, t − 1, i)
8  else
9      return SELECT(A, t + 1, r, i − j)
```

Let us denote the time complexity of SELECT on a subarray of size n as $T(n)$. We can divide the procedure into two steps,

1. finding the pivot (line 1), say it takes $T_1(n)$ time
2. partitioning and making recursive calls based on the pivot, say it takes $T_2(n)$ time.

Then, we have the following relations:

$$T(n) = T_1(n) + T_2(n)$$

For finding the pivot, let us write the time complexity:
1. Sorting each group: $\lceil n/k \rceil \times O(k^2) = O(n)$ (for $k = 5$)
2. Collecting all medians into M: $O(n)$
3. Finding the median of M: $T(\lceil n/k \rceil) = T(\lceil n/5 \rceil)$

So,

$$T_1(n) = T\left(\left\lceil \frac{n}{5} \right\rceil\right) + O(n)$$

For the rest of the procedure:
1. Partitioning A over x: $O(n)$
2. Recursing over one half of the array: $T(\max(|L_x|, |R_x|))$

Since we have shown that $|L_x| \geq \frac{3n}{10} - 6$, thus $|R_x| \leq \frac{7n}{10} + 6$, and the same upper bound also applies to $|L_x|$. So, in the worst case we will recurse on an array of size $\frac{7n}{10} + 6$ and not any larger.

So,

$$T_2(n) \leq T\left(\frac{7n}{10} + 6\right) + O(n)$$

Thus, we get the overall recurrence relation:

$$T(n) = \begin{cases} c & \text{for } n < n_0 \\ T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\frac{7n}{10} + 6\right) + O(n) & \text{otherwise} \end{cases}$$

We state a lemma (without proof) which we can directly apply here to get the O-bound for $T(n)$.

Recursing into parts less than the whole

Let $T(n)$ be a nonnegative function, and suppose there are constants c, b, n_0 such that $T(n) \leq b$ for $n < n_0$, and for all $n \geq n_0$

$$T(n) \leq T(s_1) + \cdots + T(s_\ell) + cn,$$

where ℓ is a constant, each $s_j < n$, and $s_1 + \cdots + s_\ell \leq \beta n$ for some $\beta < 1$. Then $T(n) = O(n)$.

This can be proved by induction, or by using a recurrence tree, or even by applying theorems for solving recurrence relations.

For small n, we should simply sort the entire array and select the ith element. For large enough n ($n > 70$), we have,

$$\left\lceil \frac{n}{5} \right\rceil + \frac{7n}{10} + 6 \leq \frac{n}{5} + 1 + \frac{7n}{10} + 6 \leq \frac{9n}{10} + 7 < n$$

By applying the above lemma on our recurrence, for an appropriate choice of n_0, we get that $T(n) = O(n)$, i.e. our selection algorithm runs in **deterministic worst case linear time**.[2]

[2] For a more rigorous treatment of solving this recurrence relation, see CLRS topic 9.3.

Why $k = 5$?

Let us perform a simplified analysis for the general case, where the group size is k. (Also let us assume k is odd; this leads to symmetric bounds for L_x and R_x.)

Let $m = \lceil n/k \rceil$ be the number of groups. Then, at least $\lfloor (m-1)/2 \rfloor$ group medians are less than x (and greater than x). Both counts are at least

$$\left\lfloor \frac{m-1}{2} \right\rfloor \geq \frac{m-2}{2} \geq \frac{n}{2k} - 1.$$

At most one group is partial, so discarding it leaves at least $n/2k - 2$ full groups whose median is smaller than x. In each such group, at least $(k+1)/2$ elements are smaller than x. The groups are disjoint, so

$$|L_x| \geq \frac{k+1}{2} \cdot \left(\frac{n}{2k} - 2 \right) = \frac{n}{4} + \frac{n}{4k} - (k+1) \geq \frac{n}{4},$$

where the last step holds for large n (when $n \geq 4k(k+1)$). The bound for $|R_x|$ is symmetric.

Therefore, x is an approximate median (this is a weak bound, we can get a stronger bound as well).

Then, our final recurrence can be written as

$$T(n) = T\left(\frac{n}{k}\right) + T\left(\frac{3n}{4}\right) + O(n)$$

If we pick $k = 3$, then

$$\frac{n}{3} + \frac{3n}{4} = \frac{13n}{12} > n$$

so this analysis no longer gives us a linear bound.

The smallest odd k value for which $T(n)$ is linear, is $k = 5$ (since $\frac{n}{5} + \frac{3n}{4} = \frac{19n}{20} < n$).

Worked Example

Example. Let us find the median ($i = 18$) of the following array of size $n = 35$.

34	23	3	30	20	27	24	18	33	11	12	19	6	2	10	32	35	29	22	14	26	1	28	4	7	13	21	31	15	25	16	8	17	5	9

We can arrange the array in groups of 5 as follows:

34	27	12	32	26	13	16
23	24	19	35	1	21	8
3	18	6	29	28	31	17
30	33	2	22	4	15	5
20	11	10	14	7	25	9

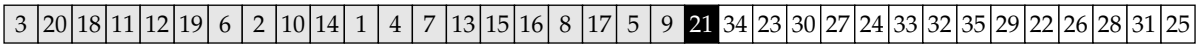
Figure 5: The array A arranged into groups of 5

Round 1 ($n = 35$, $i = 18$). Sorting each group and taking its median gives $M = (23, 24, 10, 29, 7, 21, 9)$. The median of M is its 4th smallest element, so $x = 21$.

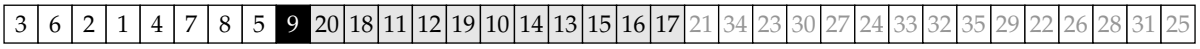


Figure 6: Each group is sorted, and the medians are computed. We recursively find the median of the medians.

Partitioning the original array by $x = 21$ gives the position of the pivot as $j = 21$. Since $i = 18$, we recurse into the left subarray:



Round 2 ($n = 20$, $i = 18$). The array is now L_x, in its original order. Its groups have medians $M = (12, 10, 7, 9)$. The median of medians is thus $x = 9$. Partitioning by x gives the position of pivot to be $k = 9$. As $i = 18 > k$, we recurse on R_x with $i \leftarrow i - j = 9$.



Round 3 ($n = 11$, $i = 9$). The groups are $(20, 18, 11, 12, 19)$, $(10, 14, 13, 15, 16)$ and the partial group (17), with medians $M = (18, 14, 17)$, so the median of medians $x = 17$. Partitioning gives the position of pivot to be $j = 8$. As $i = 9 > j$, we recurse on $R_x = (20, 18, 19)$ with $i \leftarrow 9 - 8 = 1$.



Base case ($n = 3$, $i = 1$). Sorting $(20, 18, 19)$ gives $(18, 19, 20)$, whose 1st smallest element is 18. Hence the median of the array is **18**.

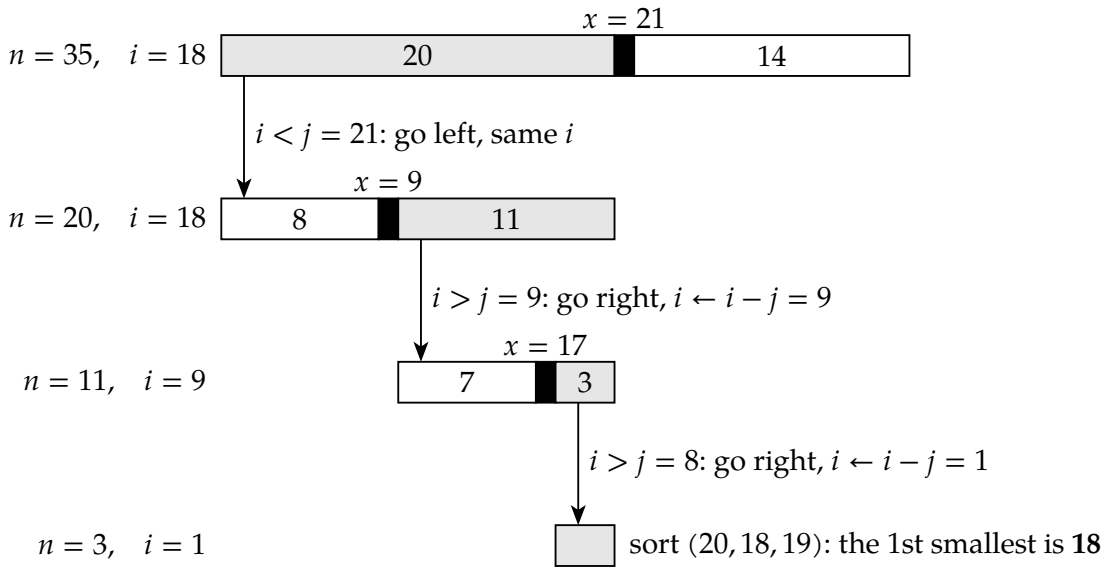


Figure 7: The recursion in the worked example. The black cell is the pivot x, the numbers are $|L_x|$ and $|R_x|$, and the shaded part is the subarray which is recursed into.

Summary

We have shown a deterministic algorithm, which runs in worst case $O(n)$ time. Note that this statement talks about asymptotic behaviour, i.e. for large n. Also, there are hidden constants associated with the time complexity. In this case, the constant is very large, due to the multiple recursive calls being made.

In practice, randomised quickselect (choosing the pivot uniformly at random) is usually much faster. Though it has a worst case time complexity of $O(n^2)$, the probability of hitting the worst case is very low, and its expected running time is $O(n)$ with a small constant. Thus, median of medians is generally not used in practice.

This algorithm is of theoretical importance, as it *guarantees* linear time in the worst case. This can be useful for safety-critical systems where a probabilistic guarantee of the running time is not enough.

Theoretically, the algorithm also shows that selection is $\Theta(n)$, since $\Omega(n)$ is needed to even read the input.

Sorting Lower Bound

We know about many sorting algorithms, such as insertion sort, bubble sort, quick sort, merge sort and heap sort. All these algorithms use only comparisons between the elements in the array to determine the final sorted order. These algorithms are called **comparison-based sorts**.

Out of the algorithms listed above, insertion sort and bubble sort run in $O(n^2)$ time, quicksort runs in $O(n^2)$ in the worst case and in $O(n \log n)$ on average. Merge sort and heap sort run in $O(n \log n)$ in the worst case.

But, can we do any better than $O(n \log n)$?

Lower Bound of a problem

Let P be a computational problem. Let n be the size of an input instance to P.

- $O(f(n))$ is said to be an **upper bound** of P, if there exists an algorithm A that solves P in $O(f(n))$ time in the worst case. Showing an upper bound involves coming up with an algorithm for P which runs in the required time.
- $\Omega(g(n))$ is said to be the **lower bound** of P, if it can be shown, that for any algorithm A that solves P, it must take at least $\Omega(g(n))$ time in the worst case. A lower bound defines a fundamental limit on the best algorithm that solves the problem P.

For any problem, advances in algorithm design gradually lowers its upper bound by coming up with better algorithms with lower worst case time complexities. On the other hand, proving a lower bound is a theoretical exercise of proving the inherent difficulty of a problem. By proving a better lower bound, the gap between the best upper and lower bounds of a problem reduces.

The central goal of algorithm design is to close the gap between the two bounds. An algorithm A is considered **asymptotically optimal** for P, if its upper bound matches the problem's lower bound. If A has a cost of $O(f(n))$ in the worst case, and $\Omega(f(n))$ is a theoretical lower bound for P, then A is optimal. Thus the two bounds converge, and the complexity of P is established to be $\Theta(f(n))$.

For our discussion, this means that $O(n \log n)$ is an upper bound for comparison-based sorts. Here, we will prove that **the lower bound for comparison-based sorts is** $\Omega(n \log n)$, thus showing the optimality of merge sort and heap sort.

The proof

Without loss of generality, assume the input array A has n distinct elements. For a comparison-based sort, we assume that it only gains information about the elements of the array by checking for comparisons between two indices in the array, a_i and a_j. By performing a sequence of such comparisons, the algorithm comes up with the correct sorted order for the array.

Under this model, any comparison-based sort may be represented as a **decision tree**. A decision tree is a full binary tree (every node having 0 or 2 children), where the non-leaf nodes represent comparison between two elements a_i and a_j, and the leaf nodes represent the final sorted sequence.

Then, the execution of the sorting algorithm on some input array A is nothing but a path from the root to a leaf of the decision tree, where at each internal node, the choice of which child to go

to is based on the outcome of the corresponding comparison. The final output of the algorithm is the sorted sequence, represented by the leaf node.

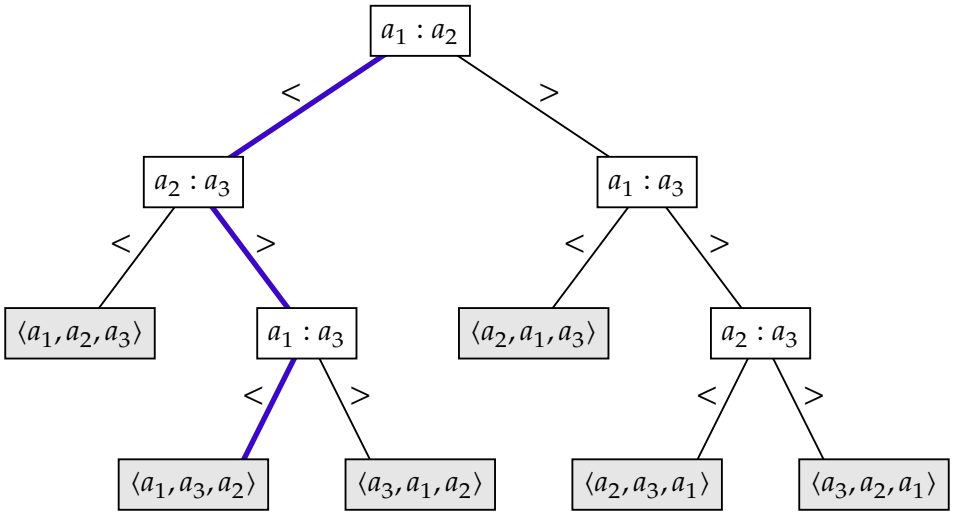


Figure 8: Decision tree of a comparison sort on $[a_1, a_2, a_3]$. Each internal node compares $a_i : a_j$, taking the left edge if $a_i < a_j$ and the right edge otherwise.

For example, suppose there exists some comparison sort S which is represented by the decision tree in Figure 8 (for $n = 3$). Let the input array $A = [2, 8, 6]$. The first comparison made is represented by the root of the tree, i.e. $a_1 < a_2$. Indeed, $2 < 8$, so we take the left branch of the tree. The next comparison is $a_2 < a_3$. Here, $8 > 6$, so we will take the right branch. The final comparison is $a_1 < a_3$ which is again true as $2 < 6$, so we take the left branch and reach the leaf node $\langle a_1, a_3, a_2 \rangle$. This represents that our final sorted sequence is $[2, 6, 8]$.

We make a few observations. Firstly, the output of the sorting algorithm is just a permutation of the input elements. This is obvious, since sorting an array is nothing but selecting the permutation in which every consecutive elements are in the correct order.

Secondly, if S is a correct sorting algorithm, then all $n!$ permutations of $[a_1, ..., a_n]$ must be present as one of the leaf nodes in its decision tree. This is because, any correct sorting algorithm must be able to produce each permutation of its input as the actual sorted sequence.

Therefore, we require that the decision tree has at least $n!$ leaves.

Now, what is the worst case time complexity of this sorting algorithm S? Since it is a comparison-based sort, we can count the number of comparisons made as its running time. In the decision tree model, this corresponds to the depth of the corresponding leaf node which represents the final sorted sequence.

Therefore, the worst case time complexity of the algorithm must be the height of the tree, h. So, we just need to get a lower bound on this height.

We know that in a binary tree of height h, the number of leaves it can have is at most 2^h. Let l be the number of leaves in the decision tree. Then, we have

$$l \leq 2^h$$

We have also shown that

$$l \geq n!$$

Thus, this implies

$$\begin{aligned} n! &\leq 2^h \\ \Rightarrow h &\geq \lg n! \\ &= \Omega(n \lg n) \end{aligned}$$

where the bound of $\lg n!$ can be obtained from the Stirling's approximation for $n!$, or using other ways as well.[3]

Since the height of the decision tree is lower bounded by $\Omega(n \lg n)$, this proves that $\Omega(n \lg n)$ is a lower bound for comparison-based sorts.

As a corollary, this also shows that **heapsort and mergesort are asymptotically optimal comparison sorts**.

[3] See *Algorithms I: Asymptotic Bounds*, Problem 1.4.5.

Problems and Solution Sketches

1. Prove that the randomized quick sort algorithm makes expected $O(n \log n)$ comparisons for every input.

Let $A = [a_1, a_2, ..., a_n]$ be the array which is to be sorted. Let $a^{(i)}$ denote the ith smallest element in A, so $a^{(1)}$ is the min element, whereas $a^{(n)}$ is the max element.

For any pair $a^{(i)}$ and $a^{(j)}$ $(i < j)$, what is the probability that they are compared?

$$..., \underbrace{a^{(i)}, ..., a^{(j)}}_{\text{block}}, ...$$

Consider the block of elements from $a^{(i)}$ to $a^{(j)}$ (in the sorted array). The first pivot that is chosen from this block decides this probability. Any pivot chosen outside this block doesn't affect this block since partitioning over such a pivot keeps the block together. If the first pivot chosen from this block is one of $a^{(i)}$ or $a^{(j)}$, then in that case during partitioning the $\left(a^{(i)}, a^{(j)}\right)$ comparison will take place. If the first pivot chosen from this block is one of the middle elements, then upon partitioning, both these elements will get split apart and never get compared. Each element in the block is equally likely to become the first pivot to be chosen from this block.

Let X_{ij} be a indicator random variable denoting that $a^{(i)}$ and $a^{(j)}$ are compared. Then

$$P\left(X_{ij} = 1\right) = \frac{2}{j - i + 1}.$$

Then, the total number of comparisons is

$$C = \sum_{i<j} X_{ij}$$

By linearity of expectation,

$$E[C] = \sum_{i<j} E\left[X_{ij}\right] = \sum_{i<j} \frac{2}{j - i + 1}$$

Group the pairs by the gap $k = j - i$. For each $k = 1, ..., n - 1$ there are exactly $n - k$ pairs (i, j) with $j - i = k$, so

$$E[C] = \sum_{k=1}^{n-1} (n - k) \cdot \frac{2}{k + 1} \leq 2n \sum_{k=1}^{n-1} \frac{1}{k + 1} = 2n \sum_{m=2}^{n} \frac{1}{m} = 2n(H_n - 1)$$

Since $H_n \leq 1 + \ln n$, we get

$$E[C] \leq 2n \ln n$$

Hence

$$E[C] = O(n \log n).$$

Alternative Solution

Since we are taking the expectation over the random pivot choices, the pivot x to be chosen is equally likely to be any of $a^{(1)}, a^{(2)}, ..., a^{(n)}$. If the pivot chosen is $a^{(i)}$, then $|L_x| = i - 1$ and $|R_x| = n - i$.

Let the number of comparisons taken by quicksort be $T(n)$, then we have the recurrence

$$T(n) = T(|L_x|) + T(|R_x|) + O(n)$$

for some specific choice of pivot x.

Taking expectation over all choices (from here, $T(n)$ denotes the expected number of comparisons), we have

$$\begin{aligned} T(n) &= \frac{1}{n}\sum_{i=1}^{n}(T(i-1)+T(n-i))+cn \\ &= \frac{2}{n}\sum_{i=0}^{n-1}T(i)+cn \end{aligned}$$

Multiply this equation by n.

$$nT(n) = 2\sum_{i=0}^{n-1}T(i)+cn^2$$

Similarly,

$$(n-1)T(n-1) = 2\sum_{i=0}^{n-2}T(i)+c(n-1)^2$$

Subtracting the two equations, we have

$$\begin{aligned} nT(n)-(n-1)T(n-1) &= 2T(n-1)+2cn-c \\ \Rightarrow nT(n) &\leq (n+1)T(n-1)+2cn \\ \Rightarrow \frac{T(n)}{n+1} &\leq \frac{T(n-1)}{n}+\frac{2c}{n+1} \end{aligned}$$

Repeatedly expanding this recurrence, one gets

$$\frac{T(n)}{n+1} \leq 2c\sum_{k=2}^{n+1}\frac{1}{k} = 2c(H_{n+1}-1)$$

We know, that $H_n = O(\ln n)$, so

$$\frac{T(n)}{n+1} = O(\log n) \Rightarrow T(n) = O(n\log n)$$

2. [CLRS] Design an algorithm to compute the second smallest of n elements with $(n + \lceil \log_2 n \rceil - 2)$ comparisons in the worst case.

Consider a comparison-based algorithm that computes the smallest element in A. Initially, any element may be potentially the smallest element. Let set S denote this set of elements which are possibly the minimum at this point.

Upon every comparison (of two elements in S), we can eliminate the larger of the two elements from S since it cannot be the minimum. Finally, we want to get to one element remaining in S, so this will take $n-1$ comparisons.

The execution of any such algorithm to find the smallest element may be represented as a **tournament tree**, where the top level nodes are the initial set S (all elements in the array), and each comparison is represented by a new node coming from the two nodes being compared, and the value of the new node is the smaller of the two parent nodes.

Consider the second smallest element in the array (call it x). It is only larger than one other element in A, which is the smallest element itself. Clearly, x was eliminated from S at some point. The only possible comparison that could have led to the removal of x must have been with the smallest element itself. This implies, that x must have been compared with the smallest element at some point.

Define P as the candidate set of second smallest elements; these are all the elements which were compared with the overall smallest element when we were finding the minimum element. Clearly, the second smallest element is just the smallest element in P. We can find this using $|P| - 1$ comparisons.

So the total number of comparisons we take to find the second smallest element is $n + |P| - 2$.

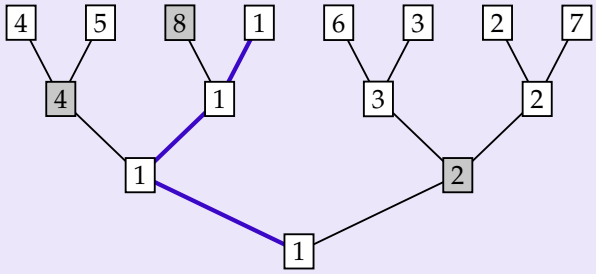


Figure 9: Tournament tree with minimal height. Shaded nodes represent P.

Now, how small can we make $|P|$? Observe, that we can consider the top nodes of the tournament tree as its leaves and the final smallest node as the root, then the tournament tree is an inverted full binary tree with n leaves. Then, we know that its height must be $\geq \lceil \log_2 n \rceil$. (We can achieve this height by comparing elements pairwise at each level, as shown in Figure 9.) We have, $|P| \leq$ height of the tree.

Thus, the total number of comparisons required to find the second smallest element in A is $n + \lceil \log_2 n \rceil - 2$.

3. Design an algorithm to compute the smallest and largest of n elements with $3\lfloor n/2 \rfloor$ comparisons in the worst case.

4. [CLRS] Prove the lower bound of $(\lceil 3n/2 \rceil - 2)$ comparisons in the worst case to find both the maximum and minimum of n numbers.

Solution sketch. Initially, all n numbers are potentially either the maximum or minimum. Let MAX be the set of numbers which are potentially the maximum and MIN denote the set of numbers which are potentially the minimum. Say we compare two elements a and b. If $a \leqslant b$, we can remove a from MAX and remove b from MIN, so we reduce the counts of both sets by 1. If we compare two elements in MIN, we reduce the size of MIN by 1, and if we compare two elements in MAX, we can reduce the size of MAX by 1. Thus, the first type of comparison is optimal. There are $\lceil n/2 \rceil$ such comparisons which can be made until MIN and MAX are disjoint, and the sets will have sizes $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$. Within each of these, only the second and third types of comparisons can be made, each reducing the set size by 1, so we require a total of $\lceil n/2 \rceil - 1 + \lfloor n/2 \rfloor - 1 = n - 2$ comparisons. Adding this to initial $\lceil n/2 \rceil$ comparisons gives $(\lceil 3n/2 \rceil - 2)$.

5. [CLRS] In the algorithm SELECT, the input elements are divided into groups of 5. Will the algorithm work in linear time if they are divided into groups of 7? Argue that SELECT does not run in linear time if groups of 3 are used.

For groups of 7,

$$T(n) = \begin{cases} c & \text{for } n < n_0 \\ T\left(\left\lceil \frac{n}{7} \right\rceil\right) + T\left(\frac{5n}{7}\right) + O(n) & \text{otherwise} \end{cases}$$

which is $O(n)$.

For groups of 3,

$$T(n) = \begin{cases} c & \text{for } n < n_0 \\ T\left(\left\lceil \frac{n}{3} \right\rceil\right) + T\left(\frac{2n}{3}\right) + O(n) & \text{otherwise} \end{cases}$$

which is superlinear.

6. [CLRS] Suppose that you have a "black-box" worst-case linear-time median subroutine. Give a linear-time algorithm that solves the selection problem for an arbitrary order statistic.

The median is the best pivot element that can be chosen for the SELECT procedure, since it divides the input array into equal partitions, so the part which we recurse into is of size at most $n/2$. So, we modify the SELECT procedure, to use the 'black-box' median subroutine to find the pivot x. The rest of the procedure remains the same.

Suppose the time complexity of this algorithm is $T(n)$, then it is decomposed into
1. Invoking the median subroutine: $O(n)$
2. Partitioning the array: $O(n)$
3. Recursively calling on one of the partitions: $T\left(\frac{n}{2}\right)$

Thus, we have the recurrence:

$$T(n) \leq \begin{cases} O(1) & \text{if } n = 1 \\ T\left(\frac{n}{2}\right) + cn & \text{otherwise} \end{cases}$$

which clearly solves to $T(n) = O(n)$ (inductively prove $T(n) \leq 2cn$ for all n).

7. [CLRS] The k-th quantiles of an n-element set are the $k-1$ order statistics that divide the sorted set into k equal-sized sets (to within 1). Give an $O(n \log k)$-time algorithm to list the k-th quantiles of a set.

Let $i_j = \lceil jn/k \rceil$ for $j = 1, ..., k-1$. The kth quantiles are the elements of ranks $i_1 < ... < i_{k-1}$.

Define the procedure QUANTILES(A, I) which takes an array A and a list I of ranks relative to A:
1. If I is empty, return.
2. Let r be the middle element of I (the $\lceil |I| /2 \rceil$th), i.e. the median rank.
3. Find the element x of rank r in A using the worst-case linear-time SELECT, and partition A around x. Now x is in position r.
4. Recurse on the subarray left of x with the ranks of I smaller than r.
5. Recurse on the subarray right of x with the ranks of I larger than r, by decreasing each rank by r.

Initially we call the procedure with QUANTILES$(A, (i_1, ..., i_{k-1}))$. When it returns, $A[i_1], ..., A[i_{k-1}]$ are the quantiles, as required.

Time complexity: The recursion always splits the array into two disjoint parts and recurses into both. The non recursion time complexity is $O(n)$. Therefore, in the recursion tree, each level contributes $O(n)$ time complexity. At each level, the rank list halves in size. The base case is hit if the rank list is empty. Thus, the total number of levels in the recursion tree is $O(\log k)$. So, the total time complexity is $O(n \log k)$.